# Self-service platform

One command to a service that already follows the rules.

A generated service passes its own policy check with 0 errors and 0 warnings.

A deliberately broken example fails with 14 errors and 3 warnings, each with a rule ID.

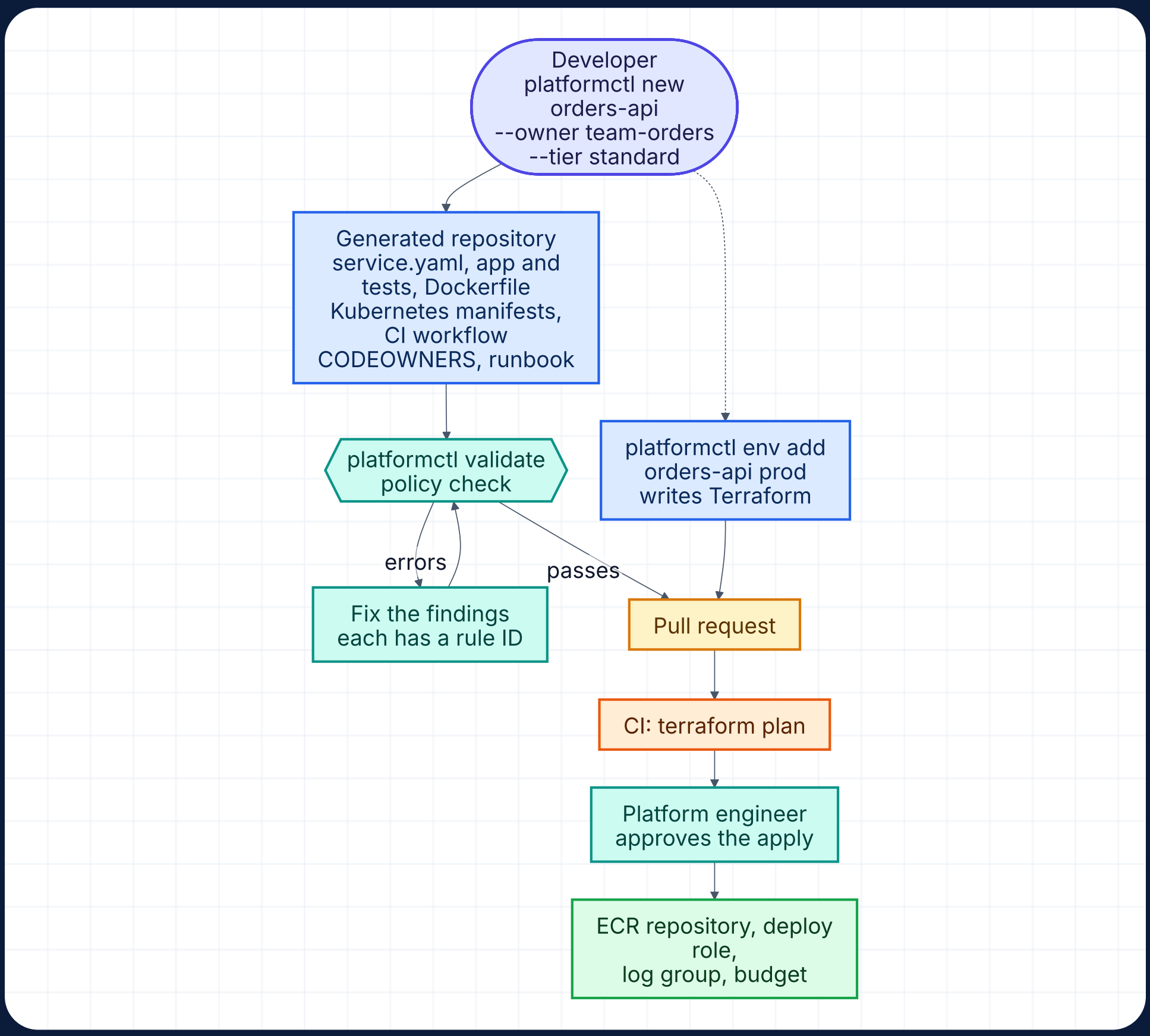
Three reliability tiers decide replicas, resources and SLO.

**Jaime Castillo Araujo**

5 schematics follow. Code: github.com/Joxejaime/projects-2026

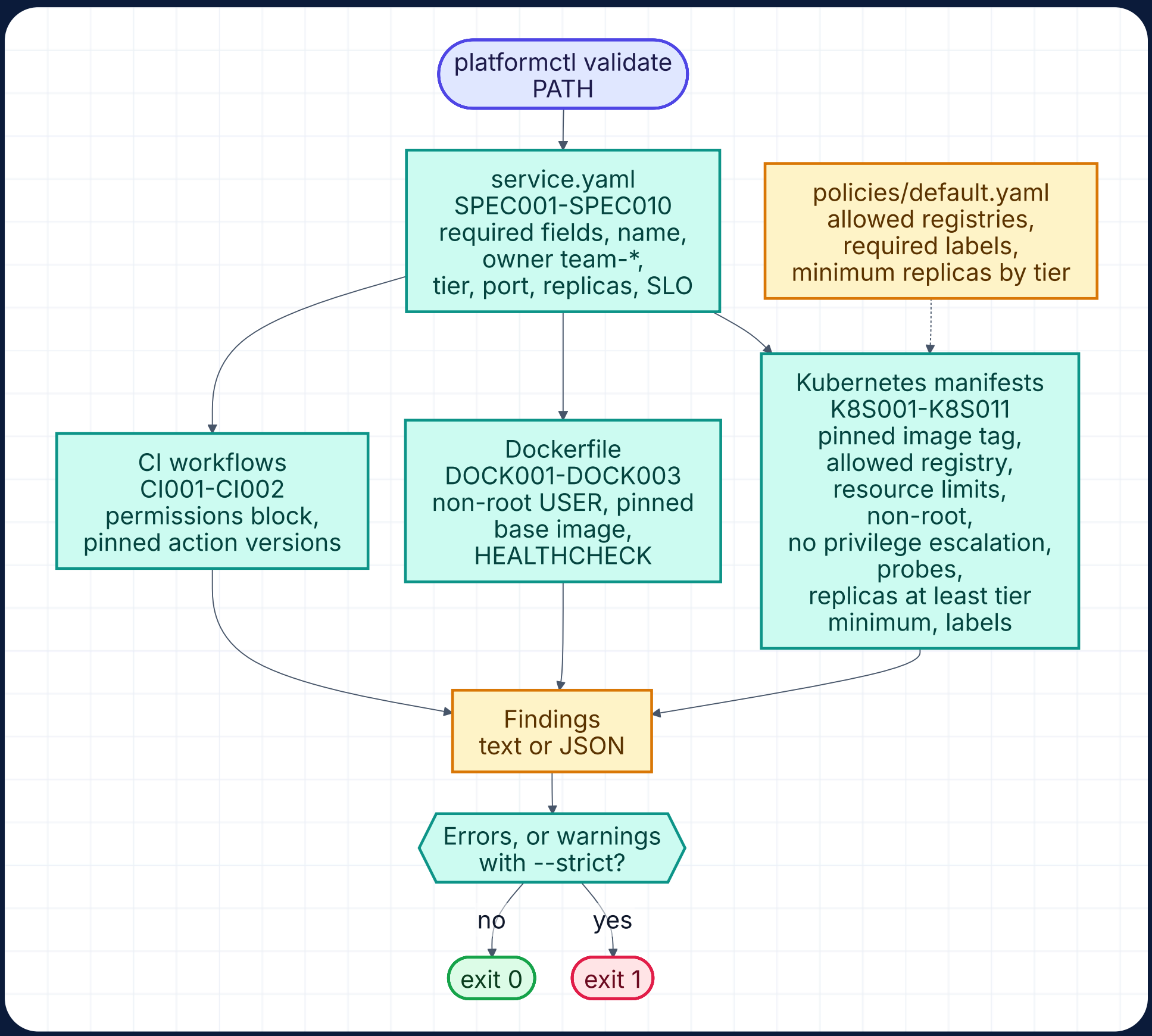
# One command to a compliant service

platformctl generates the repo, a policy check gates it, Terraform provisions the environment.



# Guardrails as code, each rule with a stable ID

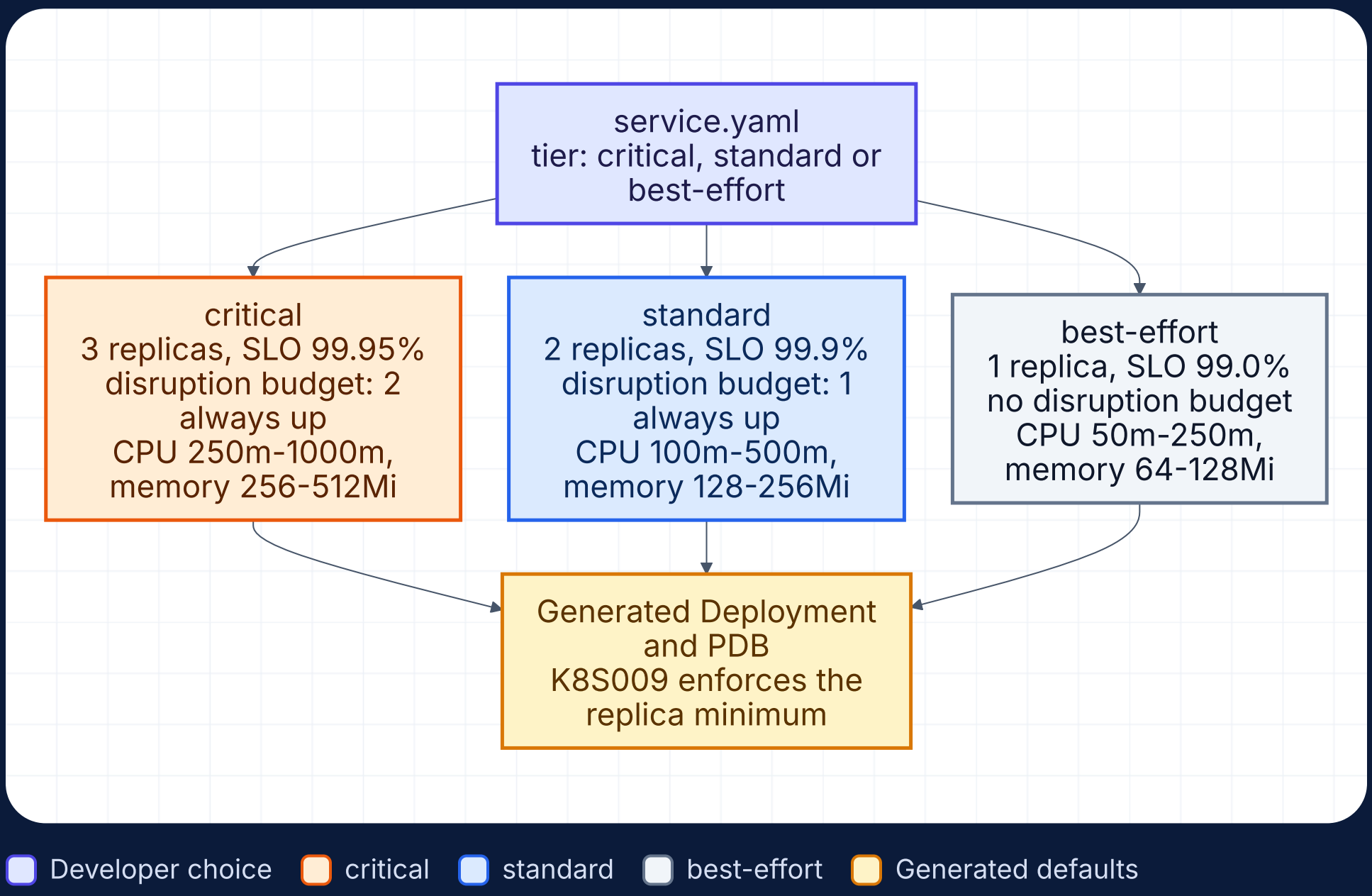
Spec, Kubernetes, Dockerfile and CI rules run on any repo. Errors fail the build.




Command Rules Policy config and findings Fails Passes

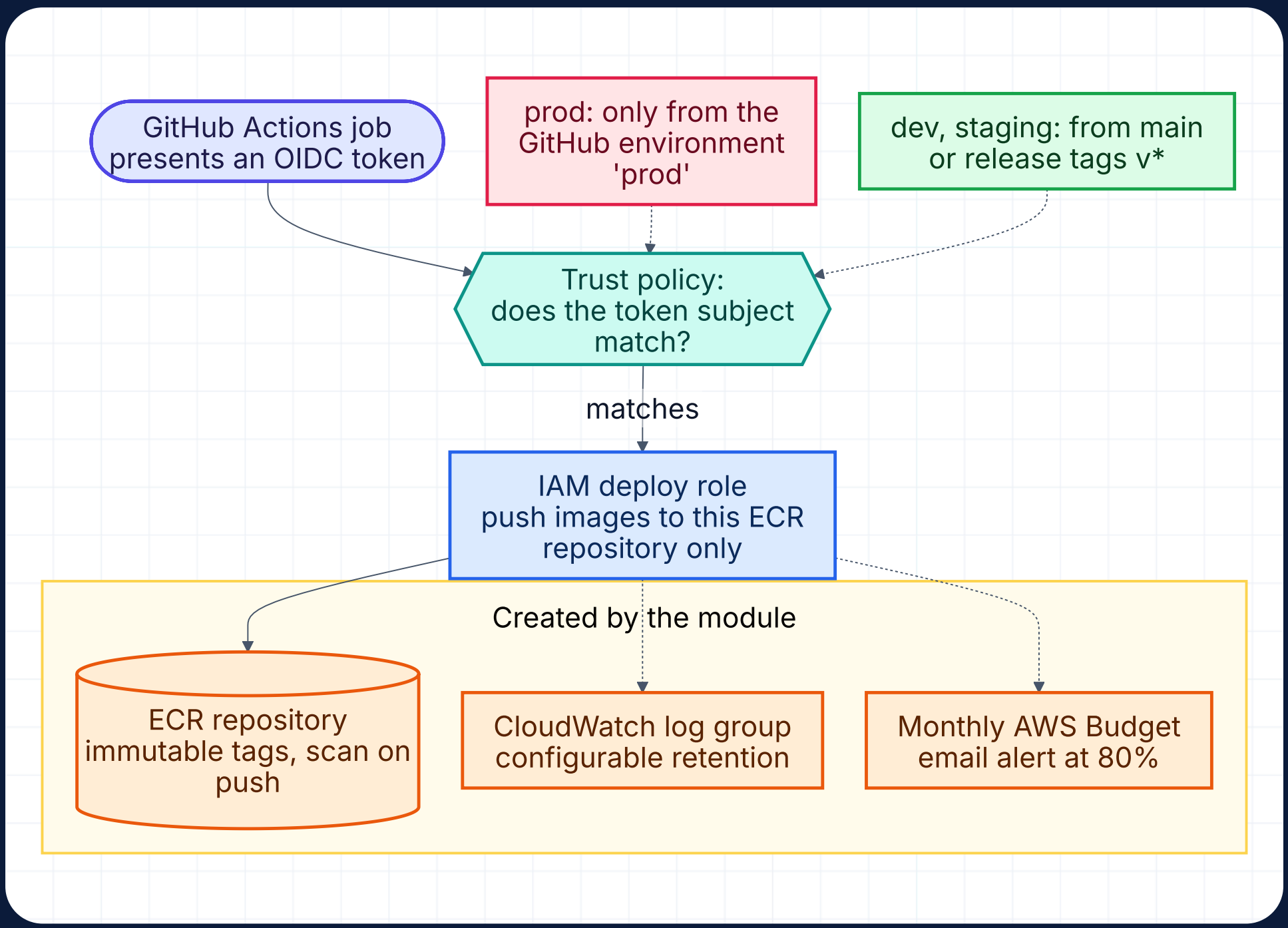
# One choice, the reliability tier, sets the defaults

critical, standard or best-effort decides replicas, resources, SLO and disruption budget.



# Deploying with no long-lived AWS keys

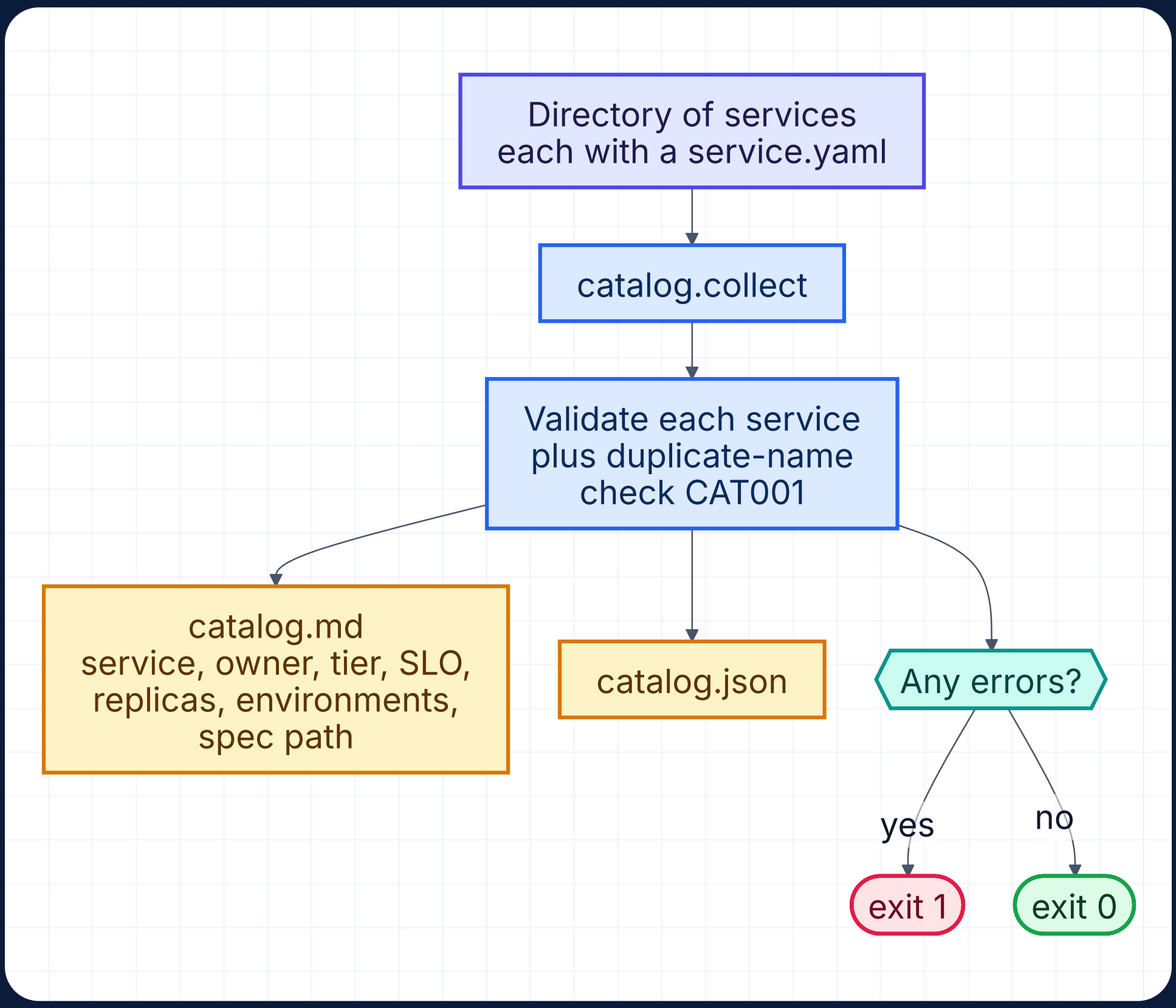
A GitHub OIDC role scoped to one ECR repository, with prod locked to a reviewed environment.




GitHub job · Trust check · Deploy role · AWS resource · Prod subject · Dev and staging subject

# A service catalog that validates as it lists

Scan every service.yaml, validate each one, and publish markdown or JSON. Errors fail CI.




Input Processing Outputs Check Fails Passes

# What I ran, and what I did not

## Run and passing

- 29 tests, including generating and validating each tier
- Policy output recorded for good and bad examples
- Service catalog built from real spec files

## Written, not yet run

- Terraform validate and apply
- The CI workflows on GitHub
- A real AWS account

**github.com/Joxejaime/projects-2026**

projects/self-service-platform, with tests, runbooks and these schematics